

DistriQuery — Literature Review

Note on citations: this draft cites papers from your Phase 1 reading list using (Author, Year) format. As flagged throughout this process, several entries (marked ⚠️ in Phase 1) still need a live verification pass before this goes into a real submission — treat this draft as structurally and argumentatively complete, but citation-unconfirmed.

1. Information Retrieval

Modern retrieval systems sit on a spectrum between lexical and semantic matching, and understanding that spectrum is a prerequisite for any RAG design decision. Classical retrieval is built on term-weighting schemes, most notably BM25 (Robertson & Zaragoza, 2009), which scores a document against a query by combining inverse document frequency with a saturating term-frequency function and a document-length normalization term. BM25 remains a strong, zero-shot, fully interpretable baseline, and its central weakness — an inability to match semantically related but lexically distinct terms — motivated the shift toward dense retrieval.

Dense Passage Retrieval (Karpukhin et al., 2020) demonstrated that a bi-encoder trained with in-batch negatives could outperform BM25 on open-domain QA by encoding queries and passages into a shared vector space and ranking by similarity. This bi-encoder pattern — independent encoding, enabling precomputed document embeddings — is what makes dense retrieval tractable at scale, in contrast to the more accurate but computationally prohibitive cross-encoder architectures used in later-stage reranking. ColBERT (Khattab & Zaharia, 2020) proposed a middle ground, using token-level "late interaction" to retain more of the cross-encoder's precision while remaining index-able, and SPLADE (Formal et al., 2021) took a different middle path, learning sparse term-expansion representations that preserve BM25-style interpretability while capturing some semantic generalization.

None of these methods scale by brute-force comparison against every stored vector. Johnson et al.'s FAISS work on billion-scale similarity search formalized the approximate nearest-neighbor (ANN) techniques — inverted-file indexing and product quantization — that underlie essentially every production vector database used in RAG systems today, including the ones under consideration for DistriQuery (Qdrant, Milvus, Weaviate, pgvector). This body of work establishes two points directly relevant to system design: first, that hybrid retrieval (combining lexical and dense signals) is not a novelty but an empirically motivated response to each method's distinct failure mode; and second, that the *choice* of ANN index structure is not a minor implementation detail but a recall/speed/memory tradeoff that should be justified against the system's actual scale and query patterns, not selected by popularity.

2. Retrieval-Augmented Generation

Retrieval-augmented generation, in its now-canonical form, was introduced by Lewis et al. (2020), who proposed jointly training a retriever and a sequence-to-sequence generator, with the model marginalizing

over top-k retrieved passages during generation. This addressed a specific weakness of purely parametric language models: their knowledge is frozen at training time and cannot be easily audited or updated. REALM (Guu et al., 2020) pursued a related but distinct idea, baking retrieval directly into language model pretraining rather than treating it as an inference-time augmentation — a more expensive approach that is less directly applicable to systems, like DistriQuery, built on inference-time retrieval over an off-the-shelf LLM.

Naive RAG, however, has well-documented failure modes that motivate most of the subsequent literature. Retrieval quality and generation faithfulness are empirically separable: a system can retrieve the correct evidence and still generate an unfaithful answer, or retrieve irrelevant evidence and still produce a plausible-sounding but ungrounded response. Additionally, LLMs exhibit positional bias in how they use long contexts — information placed in the middle of a long prompt is used less reliably than information at the beginning or end — meaning that *how* retrieved context is assembled into a prompt, not just *what* is retrieved, affects answer quality. These failure modes are the direct motivation for the "advanced RAG" techniques discussed next, and they are also why DistriQuery's evaluation framework (Phase 6) must measure retrieval and generation quality as separate metrics rather than a single end-to-end score.

3. Advanced & Agentic RAG

A first cluster of work addresses naive RAG's failure modes by adding self-correction to the pipeline. Self-RAG (Asai et al., 2023) trains a model to emit special "reflection tokens" that signal whether retrieval is needed at all, and whether generated content is actually supported by retrieved evidence — turning retrieval and self-critique into learned, integrated behaviors rather than fixed pipeline stages. Corrective RAG (Yan et al., 2024) proposes a lighter-weight variant: a separate retrieval evaluator grades retrieved documents and triggers a corrective action (such as a web-search fallback) when retrieval quality is judged insufficient. Both approaches share a common structural idea — inserting an explicit judgment step between retrieval and generation — that maps directly onto the query-planner component in DistriQuery's architecture.

A second cluster addresses questions that a single retrieval step cannot answer. Multi-hop dense retrieval (Xiong et al., 2021) retrieves iteratively, using evidence found in one hop to reformulate the query for the next, at the cost of compounding errors across hops and requiring an explicit stopping condition. GraphRAG (Edge et al., 2024) takes a structurally different approach for a different class of query — building an LLM-derived knowledge graph and hierarchical community summaries at ingestion time to support corpus-wide synthesis questions that point-retrieval methods handle poorly, at a substantially higher indexing cost.

A third cluster, agentic RAG, treats retrieval as one action among several an LLM can choose to take. ReAct (Yao et al., 2023) established the foundational pattern of interleaving explicit reasoning with actions and observations in a loop, allowing a model to decide, at each step, whether to retrieve, call a tool, or answer. Toolformer (Schick et al., 2023) addressed a related but distinct problem — how a model learns *when* a tool call is useful, via self-supervised generation and filtering of its own tool-use examples, rather than through prompting alone. Two further papers refine *when* retrieval should be triggered specifically: FLARE (Jiang et al., 2023) retrieves reactively, based on a drop in the model's own generation confidence mid-answer, while Adaptive-RAG (Jeong et al., 2024) routes queries proactively, using a trained query-complexity classifier to decide upfront between no-retrieval, single-step, and multi-step retrieval strategies. Adaptive-RAG's

proactive routing pattern is the closest existing analog to DistriQuery's own query-planner design, though — as discussed in Section 8 — none of this cluster's methods are evaluated under multi-tenant authorization constraints or explicit cost/latency budgets, both of which are first-class concerns for DistriQuery.

4. Distributed RAG Systems

This is the thinnest section of the review by necessity: the distributed-systems literature underlying DistriQuery's ingestion and serving infrastructure is largely general-purpose rather than RAG-specific, and that absence is itself informative. MapReduce (Dean & Ghemawat, 2004) established the foundational pattern of automatic parallelization and failure recovery for large-scale data processing, though its batch-oriented model is a poor fit for the low-latency, continuous ingestion DistriQuery requires — which is precisely the gap that log-structured streaming systems like Kafka (Kreps, Narkhede, & Rao, 2011) fill, offering partitioned, replicated, consumer-group-based processing that maps directly onto a "distributed workers" ingestion design. Dapper (Sigelman et al., 2010) established the distributed-tracing model — request-scoped trace and span propagation — that underlies modern observability tooling (including OpenTelemetry), and is essential once a system's request path spans multiple independent services, as an agentic RAG pipeline's does.

None of these three papers engage with retrieval quality, document versioning, or RAG-specific failure semantics at all — they are systems primitives DistriQuery borrows and adapts, not prior work it extends. This absence of literature at the intersection of distributed-systems reliability and RAG-quality evaluation is the central thread this review returns to in Section 8.

5. Multi-Tenant RAG & Security

Multi-tenancy is a well-studied database design problem, but the literature predates the retrieval architectures RAG systems depend on. Aulbach et al. (2008) compared schema-per-tenant, shared-schema, and hybrid multi-tenancy models for relational databases, establishing the isolation-versus-efficiency tradeoff that any multi-tenant system must navigate — but their schema-mapping techniques do not directly address vector-store partitioning, requiring translation rather than direct reuse for a system like DistriQuery.

LLM-integrated systems introduce a security surface that prior work does not anticipate. Greshake et al. (2023) demonstrated indirect prompt injection — adversarial instructions embedded in retrieved or tool-sourced content, rather than the user's own query, that an LLM may nonetheless obey — establishing that RAG systems must treat retrieved content as untrusted input, not as implicitly trusted context. PoisonedRAG (Zou et al., 2024) extended this threat model to the corpus itself, demonstrating that a small number of adversarially optimized documents injected into a corpus can reliably corrupt a RAG system's outputs on targeted queries. Neither paper, however, addresses the specific case DistriQuery must handle: a corpus with multiple tenants, where isolation failures can occur even absent any adversarial intent, purely as an architectural bug. This is a distinct threat category from either paper's focus, and one this review returns to as a research gap in Section 8.

Evaluating RAG systems requires separately assessing retrieval and generation quality, since the two fail independently. Retrieval quality is measured using standard information-retrieval metrics — Precision@K, Recall@K, Mean Reciprocal Rank, and NDCG — with NDCG’s support for graded (rather than binary) relevance judgments being particularly relevant to chunk-level retrieval, where a passage is rarely simply relevant or irrelevant but relevant-to-a-degree. KILT (Petroni et al., 2021) provides a shared protocol and dataset for evaluating retrieval across multiple knowledge-intensive tasks, though its reliance on a single Wikipedia snapshot limits its direct reuse for a domain-specific corpus.

Generation quality is harder to measure without expensive human annotation, motivating a cluster of LLM-as-judge frameworks. RAGAS (Es et al., 2023) decomposes generated answers into atomic claims and checks each against retrieved context to compute faithfulness, alongside answer-relevance and context-precision/recall metrics — all reference-free, requiring no hand-labeled ground truth. ARES (Saad-Falcon et al., 2024) pursues a similar goal with a different cost/reliability tradeoff: rather than relying purely on LLM judgment, it trains lightweight classifiers on synthetic data and calibrates them against a small human-labeled set using prediction-powered inference, trading setup complexity for reduced dependence on a single judge model’s biases. RGB (Chen et al., 2023) takes a different approach entirely, isolating specific RAG failure modes — noise robustness, negative rejection, information integration, counterfactual robustness — through controlled corpus perturbation, offering a methodological template (rather than a reusable dataset) for designing targeted failure-mode experiments.

A consistent limitation across this entire cluster is scope: every framework here evaluates retrieval and generation quality exclusively. None incorporate operational or systems-level metrics — latency under concurrent load, fairness of resource allocation across tenants, or quality degradation during infrastructure failures — leaving a clear methodological gap for any system, like DistriQuery, whose evaluation must span both dimensions.

7. Multimodal / Video-RAG

Extending retrieval beyond text requires a shared representation space across modalities. CLIP (Radford et al., 2021) established that contrastive pretraining on web-scale image-text pairs produces a joint embedding space enabling direct text-to-image similarity search, though this representation is frame-level rather than video-native, with no temporal modeling. ImageBind (Girdhar et al., 2023) extended this idea to six modalities — including audio — by anchoring alignment through images rather than requiring paired data for every modality combination, at the cost of weaker alignment for modality pairs (such as audio-to-text) that are only indirectly linked.

VideoAgent (Wang et al., 2024) addresses a distinct problem within video retrieval: long videos cannot be processed frame-by-frame within an LLM’s context window, so an agentic loop is used to iteratively select which frames or segments merit inspection, closely paralleling the agentic retrieval patterns discussed in Section 3, but applied to video evidence-gathering rather than text retrieval. Earlier work on temporal grounding, such as TALL (Gao et al., 2017), established the underlying task of localizing a natural-language query to a specific time span in a video, though using pre-transformer architectures unsuited to long,

unstructured content such as full lecture recordings. Whisper (Radford et al., 2022) provides the transcript-generation component this pipeline depends on — a robust, general-purpose speech-to-text system that does not itself address how transcript segments become retrievable units, which remains a downstream design decision.

Across this cluster, a consistent gap for DistriQuery’s specific use case emerges: existing Video-RAG work targets general web, egocentric, or movie video, and does not evaluate performance on structured educational content — lecture video combining speech, on-screen slides, and on-screen code or diagrams — where structural cues like slide transitions may offer a cheaper and more reliable temporal segmentation signal than the general-purpose methods surveyed here.

8. Research Gap

Synthesizing across the preceding sections reveals a consistent pattern: the literature on retrieval quality, RAG failure-mode correction, agentic control, and evaluation methodology (Sections 1, 2, 3, and 6) is extensive and mature, but assumes a corpus that is static, single-tenant, and reliably available. The literature on distributed systems reliability (Section 4) is equally mature but entirely general-purpose, with no engagement with retrieval-quality concerns at all. The multi-tenancy and security literature (Section 5) is comparatively thin and, where it exists, addresses either database schema design (predating vector retrieval) or adversarial threats to a single-tenant corpus — not the isolation guarantees a *multi-tenant* RAG system’s retrieval layer must provide.

This yields one **established gap** with direct empirical grounding in the absence observed above: no reviewed work measures how distributed-systems failure modes — ingestion-worker crashes, duplicate event delivery, stale document versions — propagate into retrieval-quality and answer-faithfulness degradation. This gap is structurally forced by the fact that Sections 4 and 6 (systems reliability and RAG evaluation, respectively) share no common literature in this review.

A second, narrower established gap concerns agentic query control under constraints: the agentic RAG literature in Section 3 uniformly assumes unconstrained access to retrieval and generation calls, with no treatment of per-tenant authorization scope or explicit cost/latency budgets as routing constraints.

A third gap — whether lecture-structured video content is well-served by general-purpose Video-RAG methods, or requires content-specific temporal segmentation — is treated here as a **hypothesized, preliminary gap**, since it depends on literature this review has not exhaustively searched, and should be verified rather than assumed before being treated as a genuine contribution.

9. Motivation for DistriQuery

The gap identified above is not an argument for building distributed infrastructure for its own sake — the project brief’s own caution against overengineering applies here. It is an argument that *evaluating* RAG techniques exclusively on static, single-tenant corpora, as the current literature does almost without exception, leaves an open empirical question with direct practical consequence: production RAG systems are

not static or single-tenant, and if retrieval quality degrades measurably under realistic ingestion failures or multi-tenant load, that degradation is currently undocumented in the literature this review surveyed.

DistriQuery is positioned to close this specific gap rather than to reproduce it: by building a system where distributed ingestion failures can be deliberately injected (Phase 6's worker-failure, duplicate-event, and ingestion-spike experiments) and measured against standard RAG-quality metrics (Section 6) using the same evaluation harness, the project can produce results that neither the systems literature nor the RAG-quality literature currently offers on its own. The primary research question that follows directly from this — how ingestion-pipeline reliability affects downstream retrieval quality and faithfulness — is deliberately scoped to be answerable within an undergraduate project's timeline, using techniques (hybrid retrieval, RAGAS-style evaluation, Kafka-based fault injection) that are each individually well-established in the literature above, combined in a way that, as best this review can determine, has not been jointly evaluated before.